

ECO387C - Fall 2024

Discussion 4

Raymond He & Husang Kim

1 Overview

Dynamic programming is widely used in theoretical economics, particularly in macro. Consider the canonical neoclassical growth model, where a central planner solves the following maximization problem:

$$\max_{\{c_t\}_{t=0}^{\infty}} \sum_{t=0}^{\infty} \beta^t u(c_t)$$

subject to an initial arbitrary k_0 , a non-negativity constraint on capital $k_t \geq 0$, and the resource constraint:

$$c_t + k_{t+1} = f(k_t) + (1 - \delta)k_t.$$

When written **sequentially**, we will eventually find the consumption-Euler equation:

$$u(c_t) = \beta (f'(k_{t+1}) + 1 - \delta) u'(c_{t+1}).$$

This **characterizes** the solution, i.e. we know that the solution has to satisfy the consumption-Euler equation, but it's not **the** solution. When we recast this problem recursively and use dynamic programming, we can often go further (and answer much richer problems).

The general strategy for finding the consumption-Euler equation is the following recipe:

1. Identify the state and control variables.
2. Set up the Bellman equation.
3. Obtain first-order-conditions by taking the derivative w.r.t. the **control** variables.
4. Invoke the Envelope theorem by taking the derivative w.r.t. the **state** variables.

When applied to the neoclassical growth model:

1. The state variable is k_t , and the control variables are c_t and k_{t+1} . To solve the problem, we need to find:

- The value function $v(k_t)$, which is the value of behaving optimally, taking the state variable as given.
- The policy functions for next period's capital k_{t+1} and consumption c_t .

However, note that we only need to solve for one of the two control variables, since the other can be pinned down by the resource constraint.

2. The Bellman equation for this problem is:

$$v(k) = \max_x \{u(k^\alpha - (1 - \delta)k - k') + \beta v(x)\}$$

where $x = g(k)$, a dummy variable indexing possible choices for k' .

While we don't know what form the value function takes explicitly, we do know that it satisfies the Bellman equation above. We will use this to solve for the value function $v(k)$.¹

Note that we are able to drop the t subscript because this is a **stationary problem**. In other words, the problem that the agent solves stays the same in each period, so there's nothing different between $t = 0$ and $t = 1$.²

3. Treating $v(x)$ as known, the RHS of the Bellman equation implies the following FOC:

$$u'(f(k) - x) = \beta v'(x)$$

However, we don't know $v(x)$, so we also don't immediately know what $v'(x)$ is.

4. Use the Envelope theorem:

Substituting the in $x = g(k)$ into the Bellman equation:

$$v(k) = u(f(k) - g(k)) + \beta v(g(k))$$

Taking the total derivative w.r.t. the state variable gives:

$$\begin{aligned} v'(k) &= u'(f(k) - x)f'(k) - u'(f(k) - x)\underbrace{g'(k)}_{=0} + \beta \underbrace{g'(k)}_{=0}v'(x) \\ &= u'(f(k) - k')f'(k) \end{aligned}$$

We know that $g'(k) = 0$ by the first order condition.

In other words, the Envelope theorem tells us that the total derivative of the value function w.r.t. k is given by the partial derivative of the objective function w.r.t. k **evaluated at the optimum**.

¹To reduce the notational burden, we'll assume full depreciation, or $\delta = 1$, from now on.

²An example of a problem that is NOT stationary is if this problem was FINITE horizon.

Evaluating this at the optimum $g(k)$ yields:

$$v'(g(k)) = u'(\underbrace{f(g(k)) - g(g(k))}_{c'})f'(g(k))$$

Combining this with the first order condition gives us the consumption-Euler equation:

$$u'(c) = \beta f'(k')u'(c')$$

While highly stylized, this problem is extremely useful as laboratory for testing numerical algorithms because it under a certain parameterization (full depreciation, i.e. $\delta = 1$ and log utility, i.e. $u(c_t) = \ln(c_t)$), a **closed-form** solution can be found. When deploying more sophisticated algorithms on richer problems, we can gain some confidence that it's working correctly if its numerical solution to the neoclassical growth model is the same as the analytical solution that we will find later this discussion.

2 Guess and verify

1. Guess that the value function takes the following form:

$$v(k) = E + F \ln(k)$$

2. This guess implies that:

$$v(k) = \max_{\{k'\}} \{ \ln(k^\alpha - k') + \beta [E + F \ln(k')] \}$$

3. Taking the derivative and rearranging yields:

$$k' = \frac{\beta F}{1 + \beta F} k^\alpha$$

4. Substituting this in and more rearranging further yields

$$\begin{aligned} v(k) &= \ln \left(k^\alpha - \frac{\beta F}{1 + \beta F} k^\alpha \right) + \beta \left(E + F \ln \left(\frac{\beta F}{1 + \beta F} k^\alpha \right) \right) \\ &= \underbrace{\beta E + \beta F \ln(\beta F) - (1 + \beta F) \ln(1 + \beta F)}_E + \underbrace{\alpha(1 + \beta F)}_F \ln(k) \end{aligned}$$

5. Finally, equating the coefficients and solving for E and F yields:

$$\begin{aligned} E &= \frac{1}{1 - \beta} \left[\ln(1 - \alpha\beta) + \frac{\alpha\beta}{1 - \alpha\beta} \ln(\alpha\beta) \right] \\ F &= \frac{\alpha}{1 - \alpha\beta} \end{aligned}$$

3 Value function iteration

As noted above, the solution to the dynamic programming problem are two functions: the value function and the policy function. It turns out that if we iterate on the equation

$$V_{j+1}(k) = \max_{k' \geq 0} \{u(k^\alpha - k' + k(1 - \delta)) + \beta V_j(k')\}$$

starting from an arbitrary starting function $V_0(k)$, we converge to the true solution (a fixed point). This is called value function iteration. To illustrate this method, assume that $\delta = 1$ and $u(c) = \ln(c)$. Then

$$V_{j+1}(k) = \max_{k' \geq 0} \{\ln(k^\alpha - k') + \beta V_j(k')\}.$$

We now go through the different iterations.

1. Set $V_0(k) \equiv 0$. This is the function which is zero everywhere. We can find $V_1(k)$ by finding

$$V_1(k) = \max_{k' \geq 0} \{\ln(k^\alpha - k')\}.$$

The optimal solution is $k' = 0$. Plugging this back in gives

$$V_1(k) = \ln(k^\alpha).$$

2. Now solve the problem

$$\begin{aligned} V_2(k) &= \max_{k' \geq 0} \{\ln(k^\alpha - k') + \beta V_1(k')\} \\ &= \max_{k' \geq 0} \{\ln(k^\alpha - k') + \beta \ln((k')^\alpha)\} \end{aligned}$$

Now the FOC is

$$\frac{1}{k^\alpha - k'} = \beta \frac{1}{(k')^\alpha} \alpha (k')^{\alpha-1}$$

or

$$k' = \frac{\beta \alpha}{1 + \beta \alpha} k^\alpha$$

Hence,

$$\begin{aligned} V_2(k) &= \ln\left(\frac{1}{1 + \beta \alpha} k^\alpha\right) + \beta \alpha \ln\left(\frac{\beta \alpha}{1 + \beta \alpha} k^\alpha\right) \\ &= \ln\left(\frac{1}{1 + \beta \alpha}\right) + \beta \alpha \ln\left(\frac{\beta \alpha}{1 + \beta \alpha}\right) + (1 + \beta \alpha) \ln(k^\alpha) \end{aligned}$$

3. Next solve

$$\begin{aligned} V_3(k) &= \max_{k' \geq 0} \{\ln(k^\alpha - k') + \beta V_2(k')\} \\ &= \max_{k' \geq 0} \left\{ \ln(k^\alpha - k') + \beta \left[\ln\left(\frac{1}{1 + \beta \alpha}\right) + \beta \alpha \ln\left(\frac{\beta \alpha}{1 + \beta \alpha}\right) + (1 + \beta \alpha) \ln((k')^\alpha) \right] \right\} \end{aligned}$$

The first order condition is

$$\frac{1}{k^\alpha - k'} = \beta(1 + \beta\alpha) \frac{1}{(k')^\alpha} \alpha (k')^{\alpha-1}$$

or

$$k' = \frac{\beta\alpha(1 + \beta\alpha)}{1 + \beta\alpha(1 + \beta\alpha)} k^\alpha$$

4. ...

5. We can see where this is going. In the limit, we will have

$$k' = \frac{\beta\alpha(1 + \alpha\beta + (\alpha\beta)^2 + \dots)}{1 + \beta\alpha(1 + \alpha\beta + (\alpha\beta)^2 + \dots)} k^\alpha = \beta\alpha k^\alpha$$

We can then write

$$\begin{aligned} \ln(k_t) &= \ln(\beta\alpha k_{t-1}^\alpha) = \ln(\beta\alpha) + \alpha \ln(k_{t-1}) \\ &= \ln(\beta\alpha) + \alpha \ln(\beta\alpha) + \alpha^2 \ln(k_{t-2}) \\ &= \ln(\beta\alpha) + \alpha \ln(\beta\alpha) + \alpha^2 [\ln(\beta\alpha) + \alpha \ln(k_{t-3})] \\ &= (1 + \alpha + \alpha^2) \ln(\beta\alpha) + \alpha^3 \ln(k_{t-3}) \\ &= (1 + \alpha + \alpha^2 + \dots + \alpha^{t-1}) \ln(\beta\alpha) + \alpha^t \ln(k_0) \\ &= \frac{1 - \alpha^t}{1 - \alpha} \ln(\beta\alpha) + \alpha^t \ln(k_0) \end{aligned}$$

To find the associated value function, plug in $\ln(k_t)$:

$$\begin{aligned} V(k) &= \sum_{t=0}^{\infty} \beta^t \ln(k_t^\alpha - k_{t+1}) = \sum_{t=0}^{\infty} \beta^t \ln((1 - \beta\alpha) k_t^\alpha) \\ &= \sum_{t=0}^{\infty} \beta^t \left[\ln((1 - \beta\alpha)) + \alpha \left\{ \frac{1 - \alpha^t}{1 - \alpha} \ln(\beta\alpha) + \alpha^t \ln(k) \right\} \right] \\ &= \sum_{t=0}^{\infty} \beta^t \left[\ln((1 - \beta\alpha)) + \frac{\alpha}{1 - \alpha} \ln(\beta\alpha) - \frac{\alpha}{1 - \alpha} \alpha^t \ln(\beta\alpha) + \alpha \alpha^t \ln(k) \right] \\ &= \frac{\ln((1 - \beta\alpha)) + \frac{\alpha}{1 - \alpha} \ln(\beta\alpha)}{1 - \beta} + \left[\alpha \ln(k) - \frac{\alpha}{1 - \alpha} \ln(\beta\alpha) \right] \frac{1}{1 - \alpha\beta} \\ &= \frac{\ln((1 - \beta\alpha))}{1 - \beta} + \frac{1}{1 - \beta} \frac{\alpha}{1 - \alpha} \ln(\beta\alpha) \left[\frac{\beta(1 - \alpha)}{1 - \alpha\beta} \right] + \frac{\alpha}{1 - \alpha\beta} \ln(k) \\ &= \frac{1}{1 - \beta} \left[\ln((1 - \beta\alpha)) + \frac{\beta\alpha}{1 - \alpha\beta} \ln(\beta\alpha) \right] + \frac{\alpha}{1 - \alpha\beta} \ln(k) \end{aligned}$$

4 Other methods

You may notice that in both methods that we covered today, we never actually used the consumption-Euler equation to find the value and policy function. Indeed, there are

powerful algorithms that iterate on/use the consumption-Euler equation instead of the Bellman equation, and they can be orders of magnitude faster.

However, iterating on the Bellman equation is still extremely popular for two reasons:

1. Convergence. As long as Blackwell's sufficiency conditions are satisfied, your initial guess of the value function is **guaranteed** to eventually converge to the right solution. Faster methods do not have this guarantee and numerical convergence can be very sensitive to your initial guess.
2. Value function iteration is extremely flexible and robust to the types of problems it can solve. Throughout the semester, you will be exposed to problems where the consumption-Euler equation does not always hold (or does not exist).

Nonetheless, deriving the consumption-Euler equation is still extremely useful for verifying the accuracy of your numerical solution:

5 Euler equation residuals

Suppose that our numerical algorithm tells us our $v(k)$ and $c^* = g(k)$ has converged to whatever numerical tolerance we set. How do we know how close our approximated solution is to the true solution?

Since the Euler equation characterizes the solution, we know that the following (rear-ranged Euler equation)

$$\frac{u_c(g(k))}{u_c(g(k'))[f(k') + 1 - \delta]} - 1 = 0$$

must be true **globally** throughout the state-space. Thus, a popular numerical exercise is to calculate the **Euler-equation residuals**, or

$$\text{EER} = \frac{u_c(g(k))}{u_c(g(k'))[f(k') + 1 - \delta]} - 1,$$

at hundreds (or thousands) **non-grid** points and to plot a corresponding histogram of the decimal log of the absolute value of all these errors.

Literatures vary in what magnitude they consider close 'enough'. If your model is 'well-behaved', then an average residual equation error of approximately -6 or -7 is required, whereas if your model is numerically complex, errors of approximately -3 or -4 are acceptable.